Applications and advanced

24F. Full Application Machine Coding

Problems: 14 (Q 24.39 – Q 24.52)

Each problem lists: core state, must-have features, and what most candidates miss.

Q 24.39 Todo Application

State: `{ todos: Array<{ id, text, completed }>, filter: 'all'|'active'|'completed' }`

Must-have: Add on Enter, toggle complete, delete, filter tabs, remaining count.

What most miss: LocalStorage persistence, edit on double-click, "Clear completed", undo delete with toast.

Q 24.40 E-commerce Product Listing with Cart

Must-have: Product grid, "Add to Cart" button, cart sidebar with quantities + total, quantity +/-, remove.

What most miss: "In Cart" instead of "Add to Cart" when already added, cart badge count, empty cart state, localStorage.

Must-have: Message list, input with send, auto-scroll, sent vs received styling, simulated bot response.

What most miss: Typing indicator, scroll-to-bottom button when scrolled up, message grouping by time, connection status.

Mock WebSocket with `setTimeout` — don't waste time on actual WS setup.

Q 24.42 Memory Card / Matching Game

JAVASCRIPT Copy

```
function handleFlip(id) {
  if (flippedIds.length === 2) return;
  const newFlipped = [...flippedIds, id];
  setFlippedIds(newFlipped);

  if (newFlipped.length === 2) {
    setMoves(m => m + 1);
    const [first, second] = newFlipped;
    if (cards[first].emoji === cards[second].emoji) {
      setCards(prev => prev.map(c =>
        c.id === first || c.id === second ? { ...c, matched: true } : c
      ));
      setFlippedIds([]);
    } else {
      setTimeout(() => setFlippedIds([]), 1000);
    }
  }
}
```

What most miss: Prevent clicking same card twice, prevent clicks during flip-back delay, Fisher-Yates shuffle, win detection.

JAVASCRIPT Copy

```javascript
function minimax(board, isMaximizing) {
  const winner = checkWinner(board);
  if (winner === 'O') return 10;
  if (winner === 'X') return -10;
  if (board.every(cell => cell)) return 0;

  if (isMaximizing) {
    let best = -Infinity;
    for (let i = 0; i < 9; i++) {
      if (!board[i]) {
        board[i] = 'O';
        best = Math.max(best, minimax(board, false));
        board[i] = null;
      }
    }
    return best;
  } else {
    let best = Infinity;
    for (let i = 0; i < 9; i++) {
      if (!board[i]) {
        board[i] = 'X';
        best = Math.min(best, minimax(board, true));
        board[i] = null;
      }
    }
    return best;
  }
}
```

Q 24.44 Snake Game

State: Snake body (array of `{x,y}`), food position, direction, score, game over.

Game loop: `setInterval` at 150-200ms. Each tick: new head based on direction, check collision (wall/self), check food (grow). Prevent 180° turns.

Q 24.45 Markdown Editor with Live Preview

JSX Copy

```jsx
function MarkdownEditor() {
  const [markdown, setMarkdown] = useState('# Hello

Type **markdown** here ... ');
  return (
    <div className="editor-layout">
      <textarea value={markdown} onChange={e => setMarkdown(e.target.value
      <div className="preview">
        <ReactMarkdown remarkPlugins={[remarkGfm]}>{markdown}</ReactMarkdo
      </div>
    </div>
  );
}
```

Q 24.46 Stopwatch with Lap Functionality

JSX Copy

```jsx
function Stopwatch() {
  const [time, setTime] = useState(0);
  const [running, setRunning] = useState(false);
  const [laps, setLaps] = useState([]);
  const startRef = useRef(0);
  const rafRef = useRef(0);

  function tick() {
    setTime(Date.now() - startRef.current);
    rafRef.current = requestAnimationFrame(tick);
  }

  function start() {
    startRef.current = Date.now() - time;
    rafRef.current = requestAnimationFrame(tick);
```

```
  function stop() { cancelAnimationFrame(rafRef.current); setRunning(false
  function lap() { setLaps(prev => [...prev, time]); }
  function reset() { stop(); setTime(0); setLaps([]); }

  const format = (ms) => {
    const m = Math.floor(ms / 60000);
    const s = Math.floor((ms % 60000) / 1000);
    const cs = Math.floor((ms % 1000) / 10);
    return `${String(m).padStart(2,'0')}:${String(s).padStart(2,'0')}.${St
  };

  return (
    <div>
      <div className="display">{format(time)}</div>
      <button onClick={running ? stop : start}>{running ? 'Stop' : 'Start'
      <button onClick={lap} disabled={!running}>Lap</button>
      <button onClick={reset}>Reset</button>
      <ul>{laps.map((l, i) => <li key={i}>Lap {i+1}: {format(l)}</li>)}</u
    </div>
  );
}
```

Uses `requestAnimationFrame` — not `setInterval` (which causes 1000 re-renders/sec).

Q 24.47 Expense Tracker

Add expenses (amount, category, date). List with category filter. Total + breakdown. localStorage.

Q 24.48 Weather Dashboard

Search city → fetch API → display current + 5-day forecast. OpenWeatherMap free API. Recent searches in localStorage.

Q 24.49 Polling App

Create poll, vote, show results as bar chart. One vote per user (localStorage). Real-time updates (SSE or polling).

Q 24.50 GitHub Explorer

Search GitHub users → display profile (avatar, bio, repos, followers). List repos (stars, forks, language). GitHub REST API.

Q 24.51 Movie Search

Search movies → display cards (poster, title, year, rating). Click → detail page. OMDB or TMDB API.

Q 24.52 Quiz App

Multiple-choice with timer per question. Score tracking. Show correct answer after each. Final summary with wrong answer review. Progress indicator.

24G. Advanced Machine Coding

Problems: 8 (Q 24.53 – Q 24.60)

Senior-level problems. In interviews, build a simplified version.

Grid, cell editing, basic formula evaluation (`=SUM(A1:A3)`, `=A1+B1`). See Chapter 11 (System Design) Q11.24 for the full design.

Q 24.54 Drawing / Whiteboard App

Canvas API. `mousedown` → drawing mode, `mousemove` → draw line, `mouseup` → stop. Tools: pen, eraser, shapes, color picker, stroke width. Undo with command stack.

Q 24.55 Collaborative Cursor Tracking

Each user's cursor position broadcast via WebSocket. Render colored cursors. Throttle position updates (50ms). Show user name next to cursor.

Q 24.56 Undo/Redo System

Command pattern. Each action has `execute()` and `undo()`. History stack + redo stack. Apply to any app — editor, drawing, form builder.

Q 24.57 Terminal Emulator

Styled `<div>`, input prompt at bottom. Implement 5-10 commands: `help`, `clear`, `echo`, `date`, `history`, `ls`. Command history with up/down arrows.

Q 24.58 Cron Expression Builder

Visual UI building a cron expression. Dropdowns for minute, hour, day, month, weekday. Preview: "Runs every Monday at 9:00 AM". Parse existing expressions.

Q 24.59 JSON Viewer / Tree Editor

Paste JSON → collapsible tree. Key, value, type coloring (string=green, number=blue, boolean=red). Click to edit values. Copy path ("data.users[0].name") on click.

Q 24.60 Email Template Builder

Drag-and-drop blocks (header, text, image, button, divider) into canvas. Configure content and styling per block. Preview and export as HTML with inline CSS.

QUICK CHECK — no marks, just for you.

1. In the Memory Card / Matching Game solution, handleFlip opens with `if (flippedIds.length === 2) return;`. What does that early return accomplish?

- ◯ It ensures the move counter is incremented only once per pair
- ◯ It blocks additional flips while two cards are already face-up, covering the one-second flip-back delay
- ◯ It prevents the same card from being flipped twice in a row
- ◯ It clears flippedIds so a third card can be flipped immediately

Check answer

2. The Stopwatch solution drives its timer with requestAnimationFrame rather than setInterval. According to the lesson, what is the problem with using setInterval here?

- requestAnimationFrame is the only API that guarantees millisecond-exact elapsed time
- At the resolution needed for centisecond display it would trigger roughly 1000 re-renders per second
- setInterval callbacks cannot read Date.now(), so elapsed time cannot be computed

Check answer

3. In the Collaborative Cursor Tracking problem, the lesson calls for throttling. What is throttled, and at what interval?

- Cursor colour re-renders, throttled to 500ms
- The user-name labels rendered beside each cursor, throttled to 1s
- WebSocket reconnection attempts, throttled to 50ms
- Cursor position updates, throttled to 50ms

Check answer

Previous
Layout and data display

Next
Interview guide

LEARN MORE

ARTICLES & DOCS

Thinking in React

react.dev

Frontend System Desi...

VIDEOS



Quiz app — machine coding

YouTube



Cinema hall seat booking

YouTube



Multi-step form — machine coding

YouTube

Hand-picked references for this lesson.